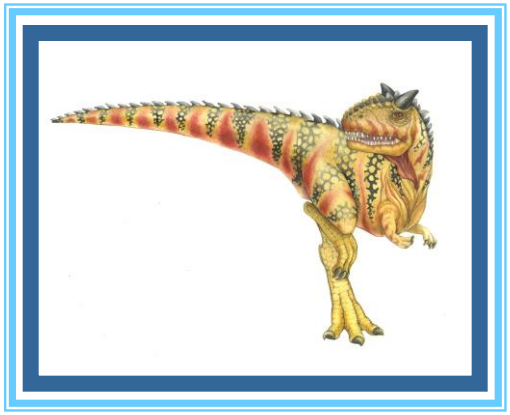
# Chapter 1: Introduction





Silberschatz, Galvin and Gagne ©2009

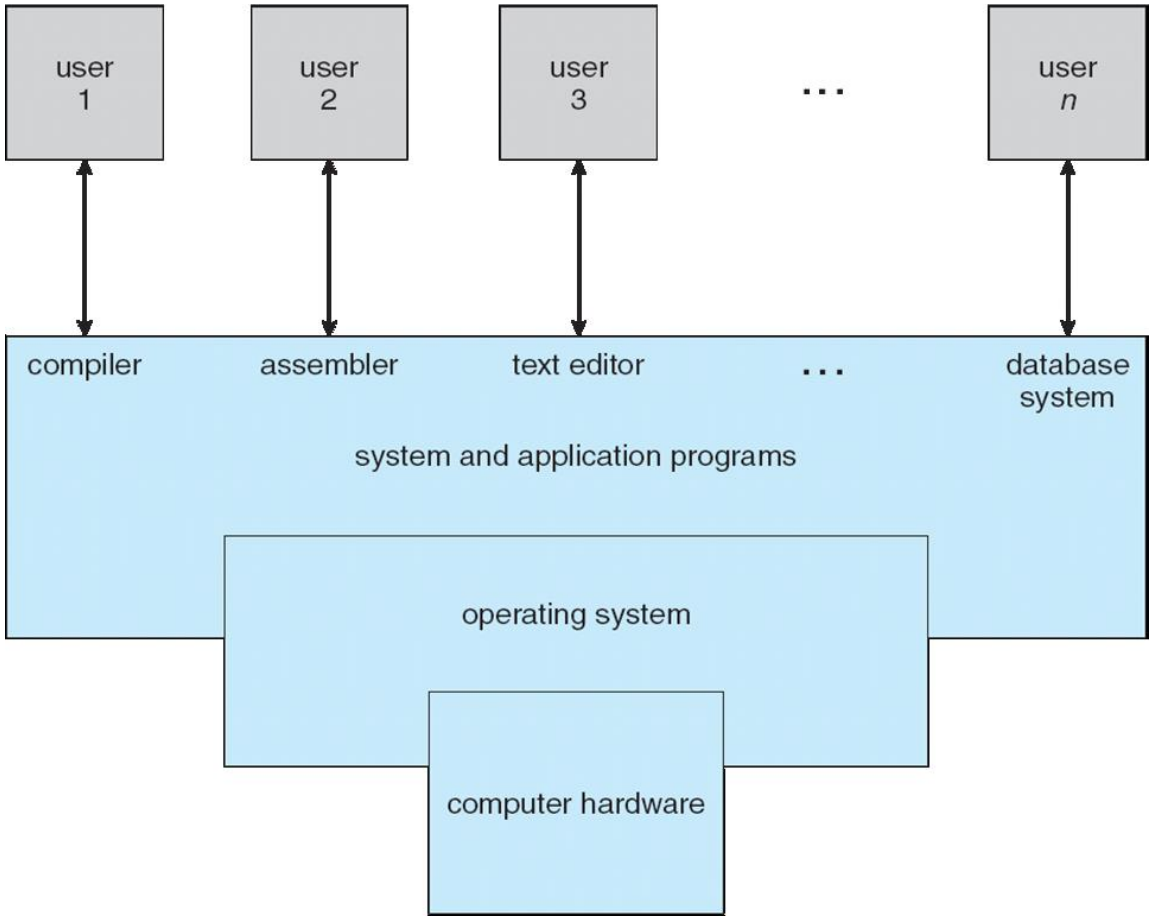
# What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

# Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

# Four Components of a Computer System



# What Operating Systems Do

- Depends on the point of view
- Users want convenience, **ease of use**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- Users of dedicated systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Handheld computers are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

# Operating System Definition

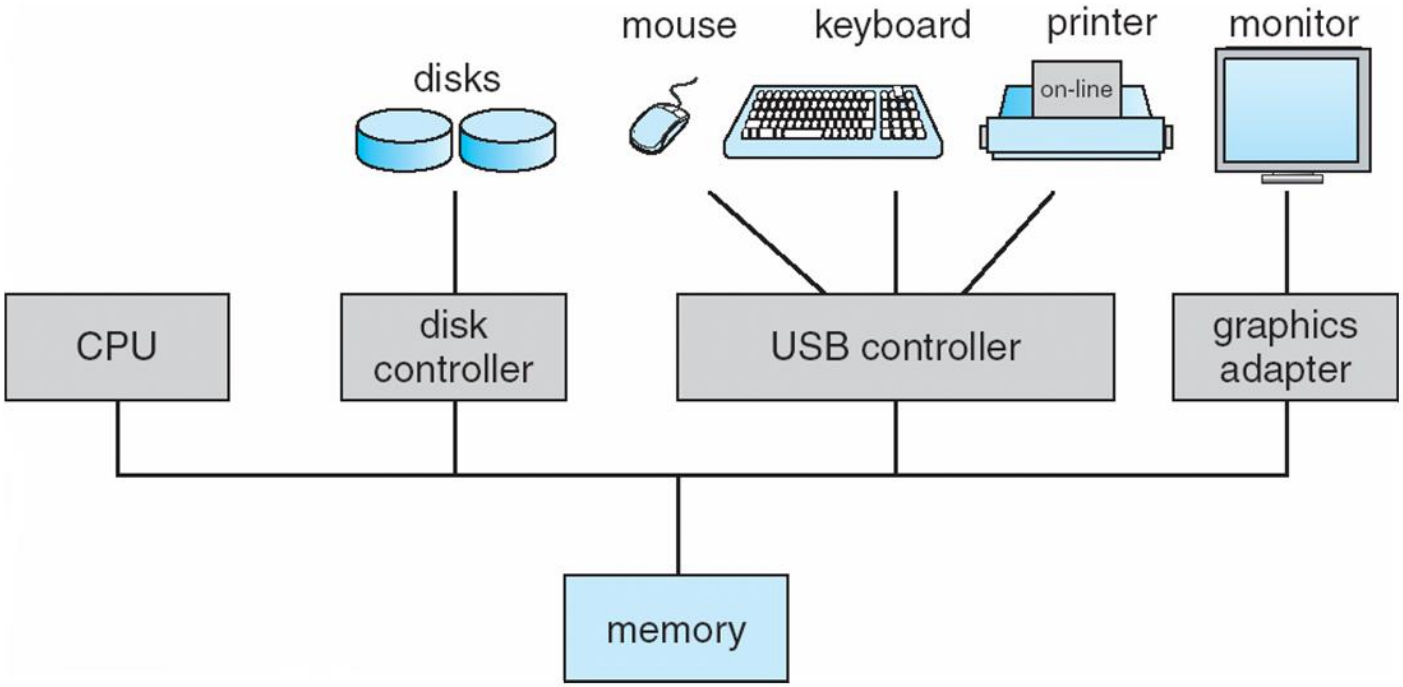
- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer
  - Special concern with operations and control of I/O devices

# Operating System Definition (Cont.)

- No universally accepted definition
- "The one program running at all times on the computer" is the **kernel**.
- Everything else is either a
  - system program
  - an application program.

# Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



# Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- Device controller informs CPU that it has finished its operation by causing an interrupt

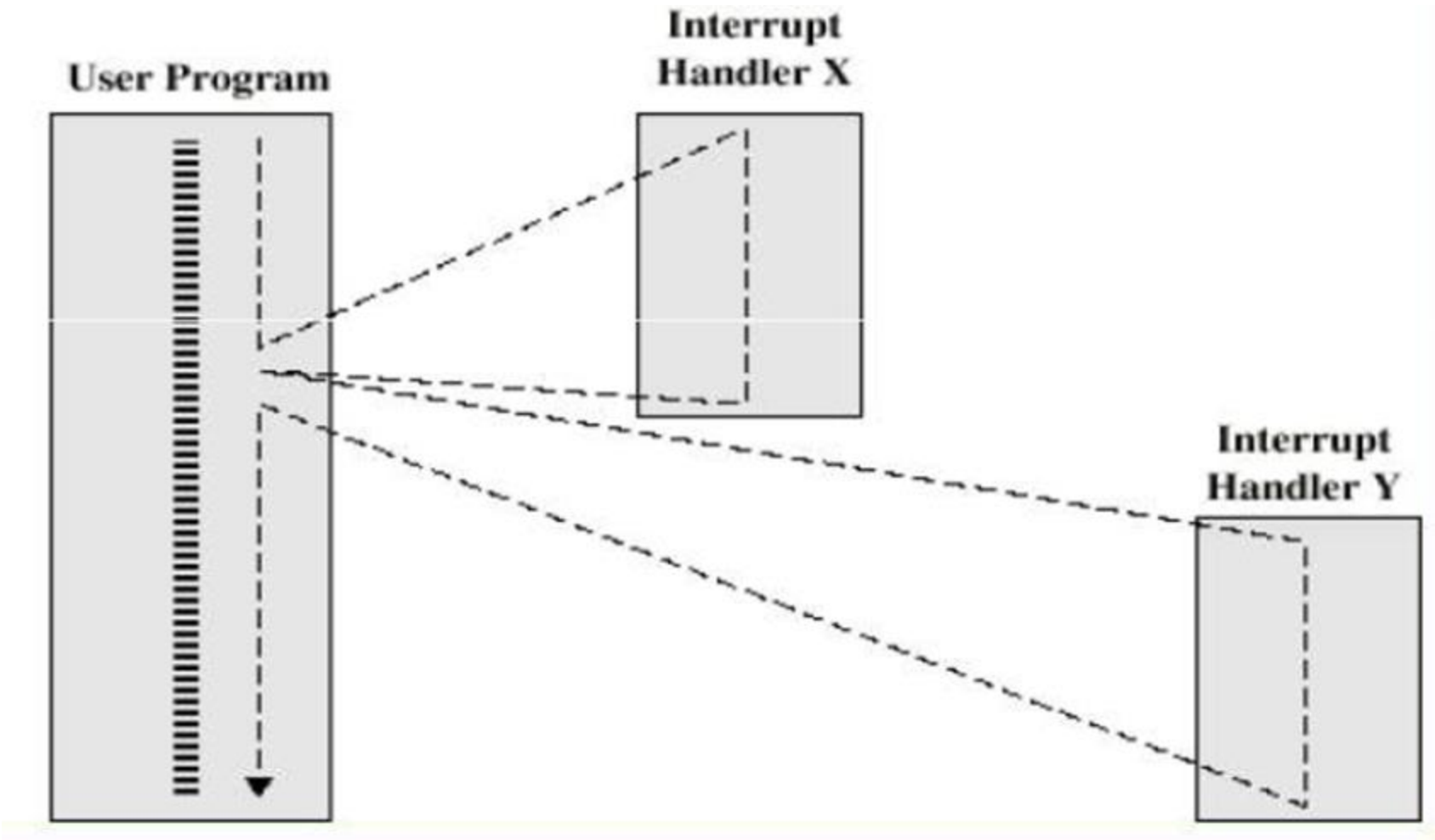
# Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- Incoming interrupts are *disabled* while another interrupt is being processed to prevent a *lost interrupt*
- A *trap* is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**

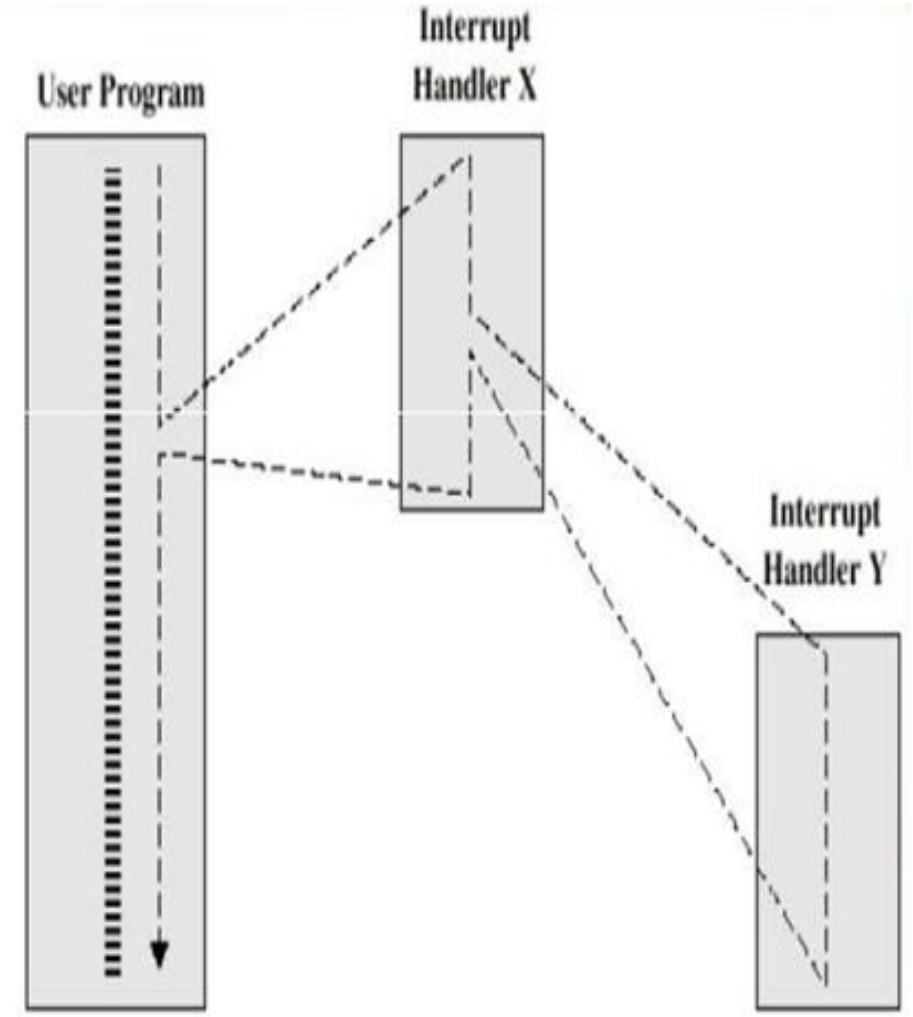
# Interrupt Handling

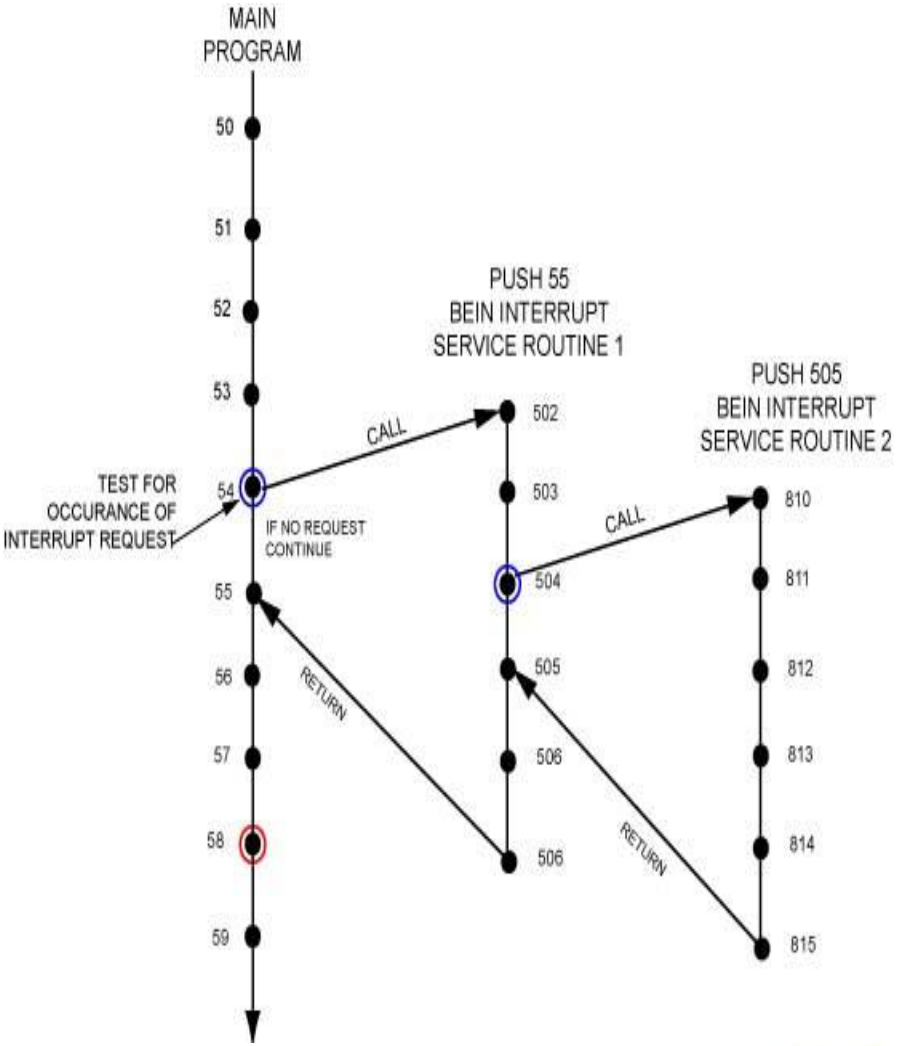
- While executing a process, an interrupt triggers.
  - It is handled by the IHR.
  - The process starts its execution again.
- However what will happen if while handling an interrupt, another interrupt triggers?
- It can be handled using two approaches
  - Sequential approach
  - Nested approach

# Sequential Interrupt Handling



# Nested Interrupt Handling



# Context Switching

- A context switch is the computing process of storing and restoring the state (context) of a CPU so that execution can be resumed from the same point at a later time.
- This enables multiple processes to share a single CPU.
- The context switch is an essential feature of a multitasking operating system.
- Context switches are usually computationally intensive and much of the design of operating systems is to optimize the use of context switches

# I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
- After I/O starts, control returns to user program without waiting for I/O completion
  - Wait instruction idles the CPU until the next interrupt
  - Wait loop
  - No simultaneous I/O processing
  - **System call** – request to the operating system to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state
  - Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt

# Storage Structure

- Main memory – only large storage media that the CPU can access directly
  - **Random access**
  - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity

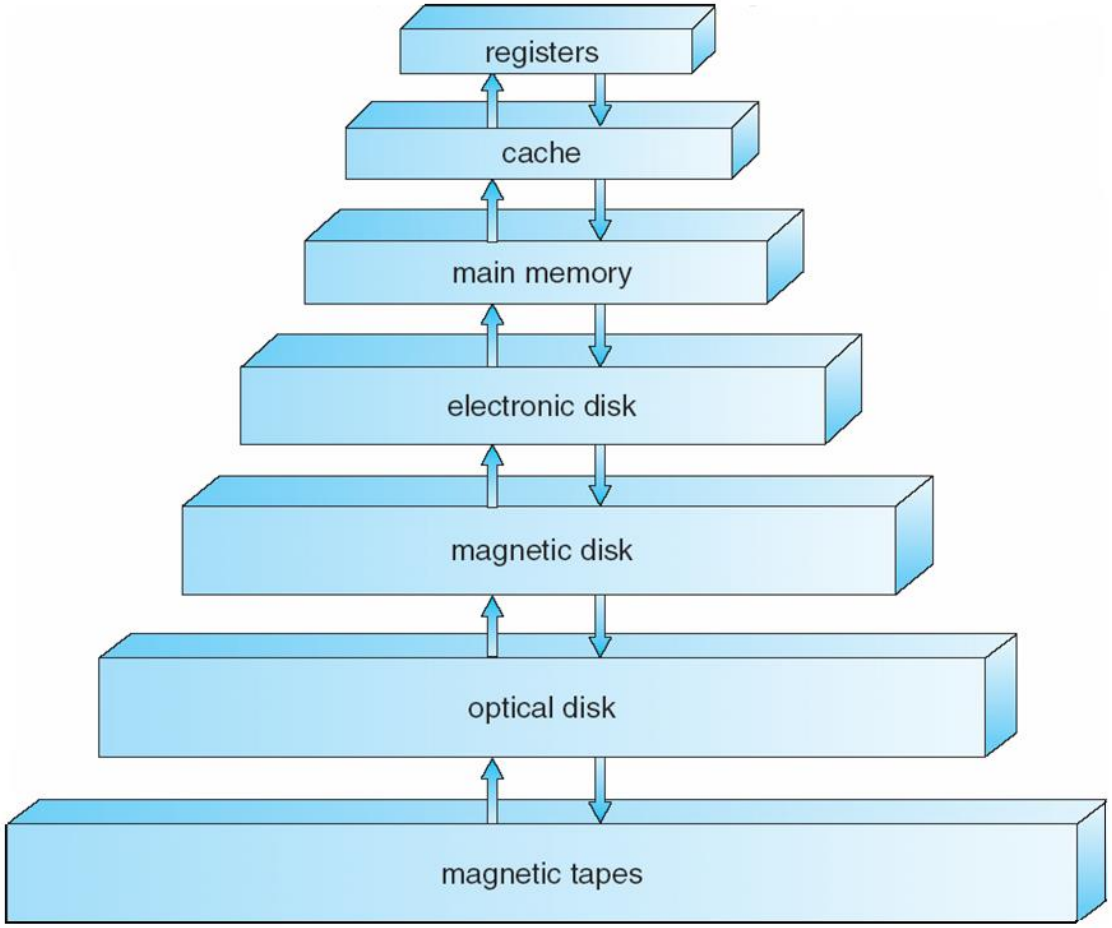
- Magnetic disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer

# Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility

- **Caching** – copying information into faster storage system; main memory can be viewed as a *cache* for secondary storage

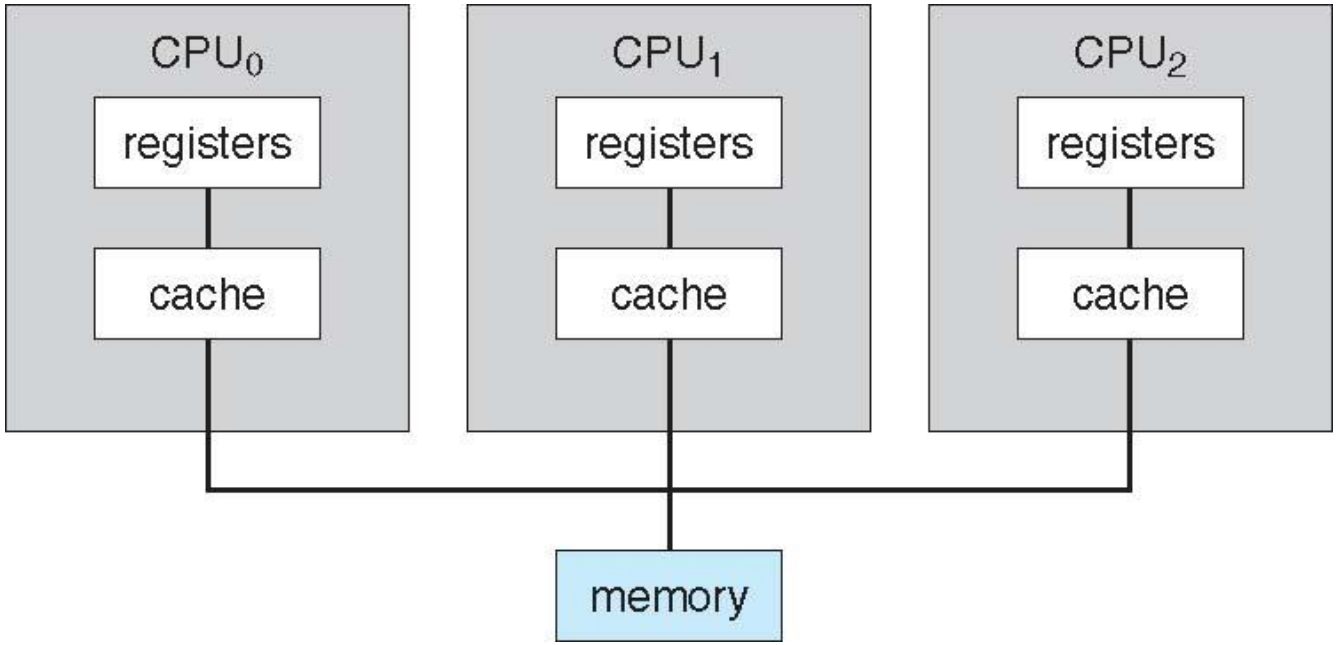
# Storage-Device Hierarchy



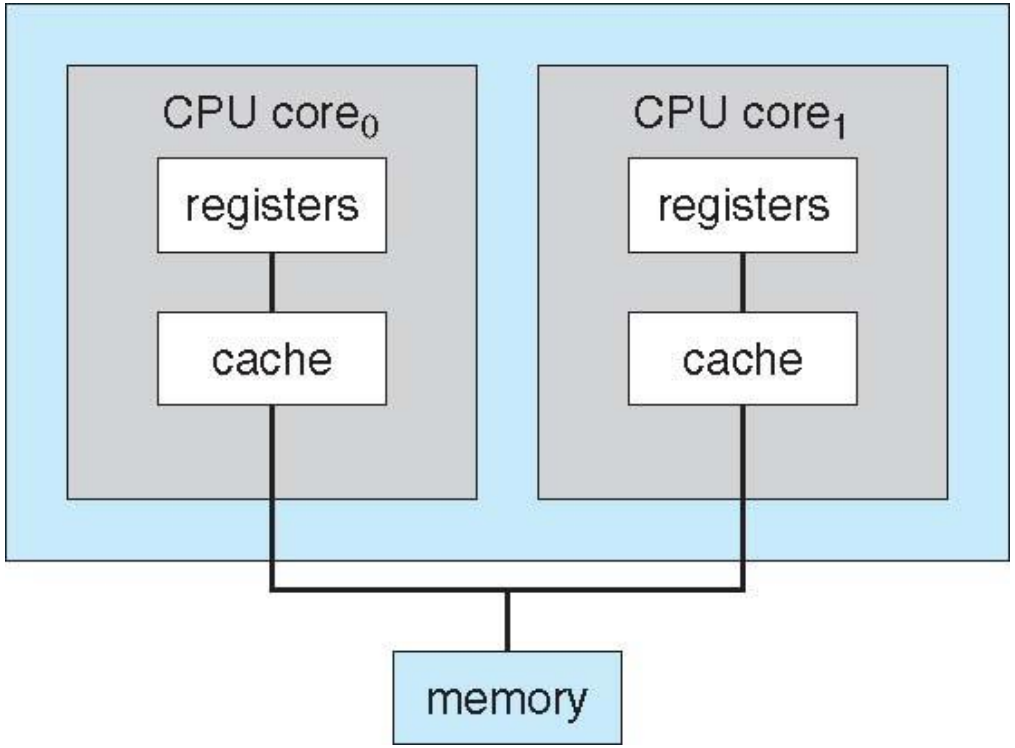
# Computer-System Architecture

- Most systems use a single general-purpose processor (PDAs through mainframes)
  - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability – graceful degradation** or **fault tolerance**
  - Two types:
    1. **Asymmetric Multiprocessing (Master-Slave)**
    2. **Symmetric Multiprocessing (Peers)**

# Symmetric Multiprocessing Architecture



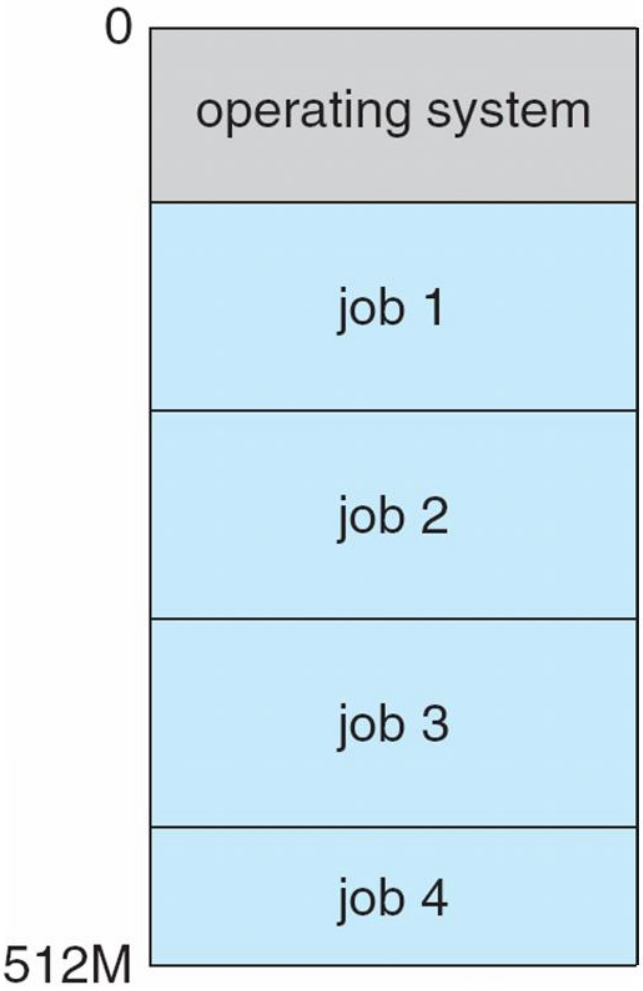
# A Dual-Core Design



# Operating System Structure

- **Multiprogramming** needed for efficiency
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so CPU always has one to execute (as a busy lawyer)
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
  - When it has to wait (for I/O for example), OS switches to another job
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory ⇨ **process**
  - If several jobs ready to run at the same time ⇨ **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run
  - **Virtual memory** allows execution of processes not completely in memory

# Memory Layout for Multiprogrammed System

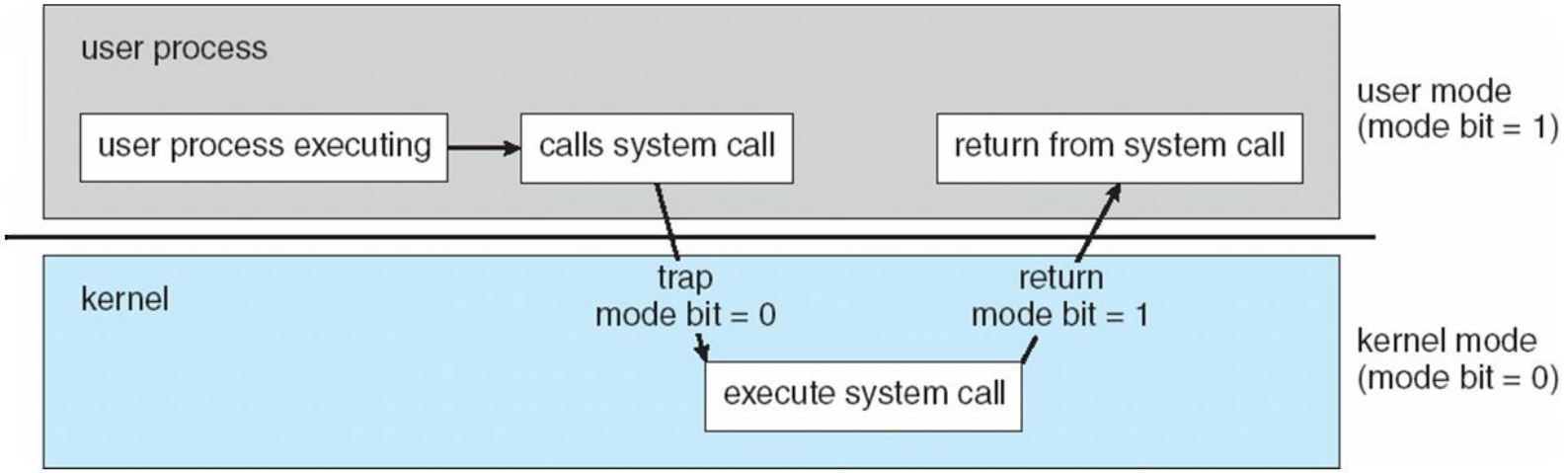


# Operating-System Operations

- Interrupt driven by hardware
- Software error or request creates **exception** or **trap**
  - Division by zero, request for operating system service
- Other process problems include infinite loop, processes modifying each other or the operating system
- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user

# Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
  - Set interrupt after specific period
  - Operating system decrements counter
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time



# Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a *passive entity*, process is an *active entity*.
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
  - Concurrency by multiplexing the CPUs among the processes / threads

# Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

# Memory Management

- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory when
  - Optimizing CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

# Storage Management

The operating system provides uniform, **logical view** of **physical** storage media.

- **Logical** storage unit — **file**.
- **Physical** storage media — disk, tapes, …

The operating system **maps** files onto physical media and access these files via the storage devices.

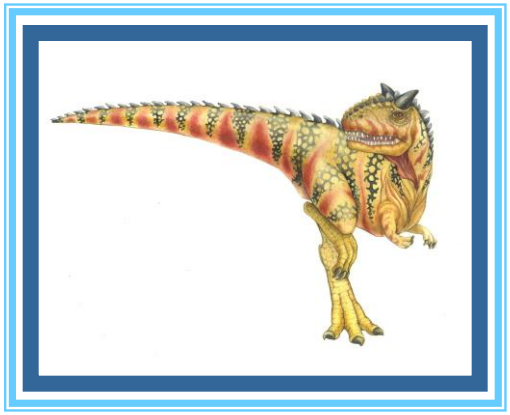
# File-System Management

- A ***file*** is a collection of related information defined by its creator.
  - Represent <u>programs</u> and <u>data</u>.
    - Data: numeric, alphabetic, or binary. free-form or non-free form.
- Files usually organized into ***directories*** to make them easier to use.
- Multiple user can access to the same file.
  - **Access control** on most operating systems to determine who can access what.
- OS activities include (chapters 10 and 11):
  - Creating and deleting files and directories.
  - Primitives to manipulate files and directories.
  - **Mapping files onto secondary storage**.
  - Backup files onto stable (non-volatile) storage media.

# Mass-Storage Management

- Computer system must provide ***secondary storage*** to back up main memory.
  - ***Disks*** are used as the principal storage medium for programs and data.
- Disks are frequently used as the source and destination of program processing.
  - Proper management is of central importance.
  - Speed of computer operation hinges on disk subsystem.
- OS activities:
  - Free-space management.
  - Storage allocation.
  - Disk scheduling.

# End of Chapter 1





Silberschatz, Galvin and Gagne ©2009